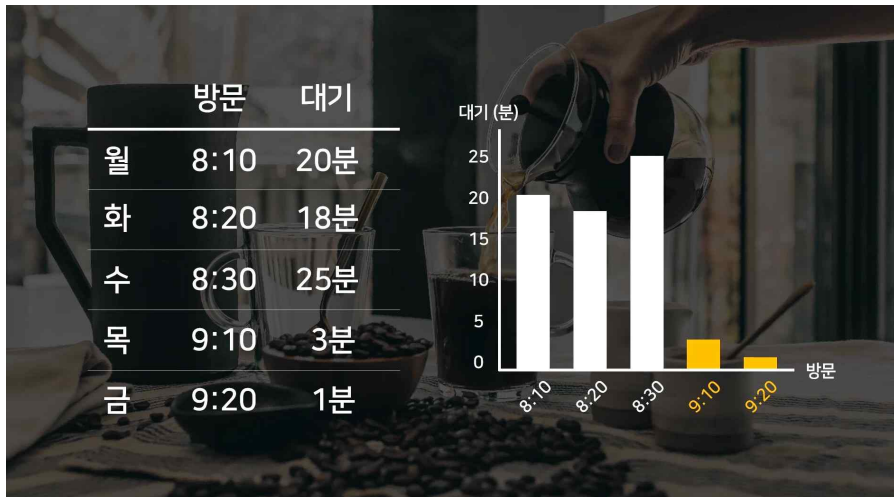


# python 데이터 분석

- 라이브러리 설치
- 데이터분석 명령어

# 1 데이터 분석 및 시각화



# 1 데이터 분석 및 시각화

---



데이터 분석



시각화

# 1 데이터 분석 및 시각화

---

데이터 분석 라이브러리 설치  
`pip install pandas`

시각화 라이브러리 설치 :  
`pip install matplotlib`

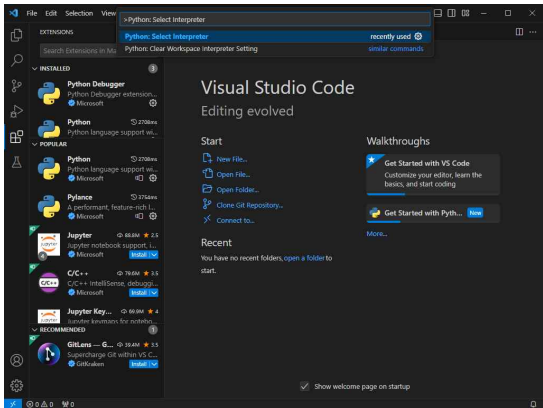
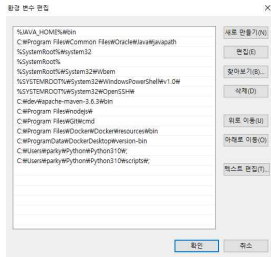
xlsx, xls 파일 열기, 데이터 확인 라이브러리 설치  
`pip install xlrd`  
`pip install openpyxl`

# 1 데이터 분석 및 시각화

[vscode에서 다른 파이썬 연결방법]

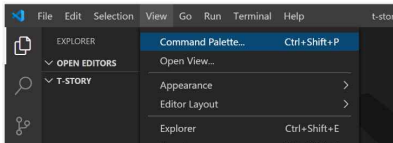
단축키Ctrl + Shift + P(Mac은 Cmd + Shift + P)

시스템 > 고급 시스템 설정 > 환경변수

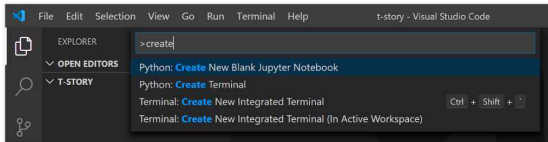


# 1 vscode 주피터 노트북 사용가능

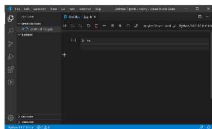
vs code에 python만 설치 되어 있으면 사용가능



1. view > command palette 클릭



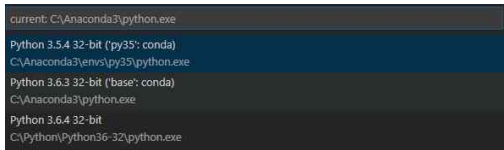
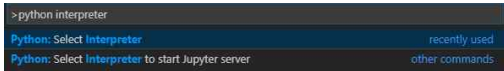
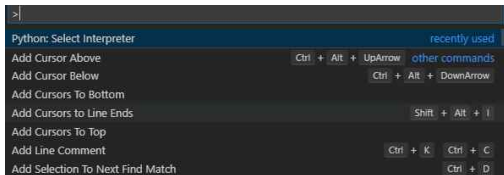
2. Python: Create New Blank Jupyter Notebook  
입력



3. 화면 출력

# 1 vscode 주피터 노트북 사용가능

vs code 에서 python 설치 위치 변경



1. F1 또는 Ctrl + Shift + P 입력(Palette 명령)

2. Python Interpreter를 입력하여 "Python: Select Interpreter"

3. 원하는 Python Interpreter를 사용

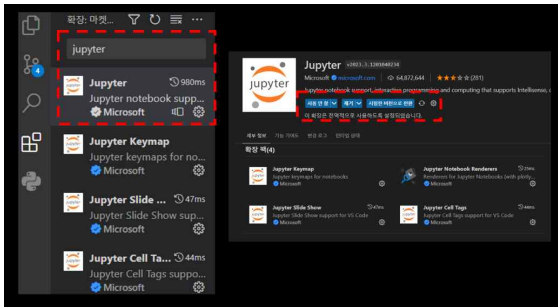
이렇게 되면 settings.json파일이 생성 혹은 변경되어 Python interpreter의 경로를 지정해주게 된다.

pip install jupyter 설치

<https://gentlesark.tistory.com/32>

# 1 vscode 주피터 노트북 사용가능

vs code 에서 python 설치 위치 변경

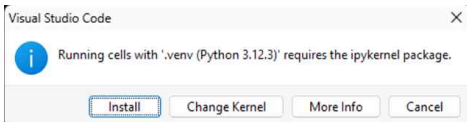


1. F1 또는 Ctrl + Shift + P 입력(Palette 명령)

2. Python Interpreter를 입력하여 "Python: Select Interpreter"

3. 원하는 Python Interpreter를 사용

이렇게 되면 settings.json파일이 생성 혹은 변경되어 Python interpreter의 경로를 지정해주게 된다.



pip install jupyter 설치

<https://gentlesark.tistory.com/32>

# 1 Series 1차원 데이터

01\_series1.py

```
# series  
# 1차원 데이터 (정수, 실수, 문자열 등)
```

```
import pandas as pd
```

```
### Series 객체 생성
```

```
# 예) 1월부터 4월까지 평균 온도 데이터 (-20, -10, 10, 20)
```

```
temp = pd.Series([-20, -10, 10, 20])
```

```
print(temp)
```

```
print(temp[0]) # 1월 온도
```

```
print(temp[2]) # 3월 온도
```

\* 데이터 타입은 pd.Series class  
\* 1차원 데이터 정렬

# 1 Series 1차원 데이터 - index 지정

01\_series1.py

```
# series
# 1차원 데이터 (정수, 실수, 문자열 등)

import pandas as pd

### Series 객체 생성 index 0,1,2,3 -> Jan, Feb, Mar, Apr 로 변경
temp = pd.Series([-20, -10, 10, 20], index=['Jan', 'Feb', 'Mar', 'Apr'])
print(temp)

print(temp['Jan']) # Index Jan (1월)에 해당하는 데이터 출력
print(temp['Apr']) # Index Apr (4월)에 해당하는 데이터 출력

# temp['Jun'] # 존재하지 않는 Index 접근 시도 시 에러
```

\* index 추가

\* 존재하지 않는 index 시도 시 에러  
\* 잘 확인을 해야 함.

## 2 DataFrame 2차원 데이터

02\_dataframe1.py

# 2. DataFrame

# 2차원 데이터 (Series 들의 모음)

# (dictionary) 자료구조를 통해 생성

import pandas as pd

```
data = {  
    '이름': ['강나래', '강태원', '강호림', '김수찬', '김재욱', '박동현', '박혜정', '승근열'],  
    '학교': ['구로고', '구로고', '구로고', '구로고', '구로고', '디지털고', '디지털고', '디지털고'],  
    '키': [197, 184, 168, 187, 188, 202, 188, 190],  
    '국어': [90, 40, 80, 40, 15, 80, 55, 100],  
    '영어': [85, 35, 75, 60, 20, 100, 65, 85],  
    '수학': [100, 50, 70, 70, 10, 95, 45, 90],  
    '과학': [95, 55, 80, 75, 35, 85, 40, 95],  
    '사회': [85, 25, 75, 80, 10, 80, 35, 95],  
    'SW특기': ['Python', 'Java', 'Javascript', "", 'C', 'PYTHON', 'C#']  
}
```

# 전체 출력

print(data)

# 이름 리스트 출력

print(data['이름'])

# 키 리스트 출력

print(data['키'])

\* data 입력 : dictionary 형태

df = pd.DataFrame(data)

- df 는 DataFrame 타입

## 2 DataFrame 2차원 데이터

02\_dataframe2.py

```
# 2. DataFrame  
# dictionary에서 DataFrame 객체 생성  
import pandas as pd
```

```
# Data 정보가 있음.
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

\* df 는 DataFrame 타입  
pandas > DataFrame 타입

## 2 DataFrame 2차원 데이터

02\_dataframe2.py

```
#2. DataFrame  
#데이터 접근  
import pandas as pd
```

# Data 정보가 있음.

```
print(df['이름'])    #이름 컬럼 출력  
print(df['키'])      #키 컬럼 출력  
print(df[['이름','키']]) #이름, 키 컬럼 출력  
print(df[['이름','키','SW특기']]) #3개 컬럼 출력
```

\* 각 컬럼으로 접근

## 2 DataFrame 2차원 데이터

02\_dataframe3.py

# 2. DataFrame

# DataFrame 객체 생성 (Column 지정)

# data 중에서 원하는 column 만 선택하거나, 순서 변경 가능

import pandas as pd

# Data 정보가 있음.

# (Column 지정)

df = pd.DataFrame(data, columns=['이름', '학교', '키'])

print(df)

df = pd.DataFrame(data, columns=['이름', '키', '학교'])

print(df)

\* 원하는 컬럼만 출력  
\* 순서도 변경가능

### 3 index 지정 – index 지정후 출력

03\_index1.py

# 3. Index

# 데이터에 접근할 수 있는 주소 값

import pandas as pd

```
data = {  
    '이름': ['강나래', '강태원', '강호림', '김수찬', '김재욱', '박동현', '박혜정', '승근열'],  
    '학교': ['구로고', '구로고', '구로고', '구로고', '구로고', '디지털고', '디지털고', '디지털고'],  
    '키': [197, 184, 168, 187, 188, 202, 188, 190],  
    '국어': [90, 40, 80, 40, 15, 80, 55, 100],  
    '영어': [85, 35, 75, 60, 20, 100, 65, 85],  
    '수학': [100, 50, 70, 70, 10, 95, 45, 90],  
    '과학': [95, 55, 80, 75, 35, 85, 40, 95],  
    '사회': [85, 25, 75, 80, 10, 80, 35, 95],  
    'SW특기': ['Python', 'Java', 'Javascript', "", 'C', 'PYTHON', 'C#']  
}
```

# (Index 지정)

df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])

print(df.index)

\* index를 지정

### 3 index 지정 - index 이름설정, 초기화, 삭제

03\_index1.py

```
import pandas as pd
```

# Data 정보가 있음.

# (Index 지정)

```
df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

```
print(df.index)
```

```
df.index.name='지원번호'
```

```
print(df)
```

# index 초기화 - vscode에는 반영이 안됨.

```
df.reset_index(inplace=True) # 이 명령어일때만 가능
```

```
print(df)
```

# index 삭제 - 원래 쓰던 '지원번호' 인덱스 삭제 - vscode에는 반영이 안됨

# 실제 반영이 된것이 아니기에 다시 출력하면 나타남

```
df.reset_index(drop=True, inplace=True) # inplace를 해야만 반영됨.
```

```
print(df)
```

\* index 이름을 지정

\* Index 이름변경

```
df.index.values[3]='new_name'
```

\* Index 이름 변경방법

# index를 지정

```
df2.index = ['no.1', 'no.2', 'no.3']
```

```
df2.rename(index={'no.1':'1번'}, inplace=True)
```

|      | 이름 | 학교 | 합계 |
|------|----|----|----|
| 지원번호 |    |    |    |

|      |     |     |     |
|------|-----|-----|-----|
| no.1 | 홍길동 | 가산고 | 100 |
|------|-----|-----|-----|

|      |     |     |    |
|------|-----|-----|----|
| no.2 | 유관순 | 가산고 | 90 |
|------|-----|-----|----|

|      |     |      |    |
|------|-----|------|----|
| no.3 | 이순신 | 다지월고 | 95 |
|------|-----|------|----|

\* Column 이름 변경방법

```
df.columns=['a','b']
```

```
df.rename(columns = {'old_nm' : 'new_nm'},  
inplace = True)
```

\* df.reset\_index() 명령어만 입력시

- vscode에는 반응이 없음

\* df.reset\_index(drop=True)

inplace=True 해야만 반영됨.

명령어만 입력시

- vscode에는 반응이 없음

### 3 index 지정 - 삭제 후 반영

03\_index1.py

```
import pandas as pd
```

```
# Data 정보가 있음.
```

```
# 실제 데이터에 바로 반영  
df.reset_index(drop=True, inplace=True)  
print(df)
```

\* inplace=True 해야지만 반영을 시킴

### 3 index 지정 – 이름으로 index지정

03\_index2.py

```
import pandas as pd
```

# Data 정보가 있음.

```
# (Index 지정)
```

```
# 지정한 column으로 index를 설정
```

```
df = pd.DataFrame(data)
```

```
df.set_index('이름', inplace=True) # 반영을 해야함
```

```
print(df)
```

\* inplace=True 해야지만  
vscode에서 반영되어 출력됨.

### 3 index 지정 – 내림차순, 오름차순 지정

03\_index3.py

```
import pandas as pd
```

```
# Data 정보가 있음.
```

```
# (Index 지정)
```

```
index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번']
```

```
df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

```
df.index.name="지원번호"
```

```
# 이름 내림차순(가나다순) 정렬.
```

```
df.sort_index(inplace=True)
```

```
# 이름 오름차순(하타파순) 정렬
```

```
df.sort_index(ascending=False, inplace=True)
```

```
print(df)
```

\* inplace=True 해야지만  
vscode에서 반영되어 출력됨.

# Quiz

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

1. 영화정보만 출력하시오.
2. Index를 영화로 지정하고,역순정렬 하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
Print(df)
```

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
```

```
data = {
```

```
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방  
의 선물', '암살'],
```

```
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
```

```
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
```

```
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
print(df['영화'])
```

# 실습 과제

## 영화

## 4 파일 저장 및 열기 - csv파일 저장

04\_filesave1.py

```
import pandas as pd
```

```
# Data 정보가 있음.
```

```
# (Index 지정)
```

```
df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

```
df.index.name = "지원번호"
```

```
# csv파일 저장하기 인코딩 없이 저장하면 excel파일에서는 한글 깨짐  
df.to_csv('score.csv', encoding='utf-8-sig')
```

```
# index번호 제외하고 저장
```

```
df.to_csv('score.csv', encoding='utf-8-sig', index=False)
```

\* encoding='utf-8-sig' 없으면  
excel에서는 한글이 깨짐

## 4 파일 저장 및 열기 - txt파일, excel로 저장

04\_filesave2.py

```
import pandas as pd
```

```
# Data 정보가 있음.
```

```
# (Index 지정)
```

```
df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

```
df.index.name = "지원번호"
```

```
# txt파일 저장하기, tab으로 구분된 텍스트 파일 저장
```

```
df.to_csv('score.txt', sep='\t')
```

\* txt 파일로 저장  
- tab으로 구분하여 저장

```
# excel파일 저장하기
```

```
df.to_excel('score.xlsx')
```

\* 엑셀파일 저장

# 4 파일 저장 및 열기 - csv 파일 열기

04\_filesave3.py

```
import pandas as pd
```

```
# csv파일 열기
```

```
df = pd.read_csv('score.csv')
```

```
# 1번째 row skip후 출력 : 이름 학교 키... 부분 삭제
```

```
df = pd.read_csv('score.csv', skiprows=1)
```

```
# 1,3,5 번째 row skip후 출력
```

```
df = pd.read_csv('score.csv', skiprows=[1,3,5])
```

```
# 지정된 개수 만큼 출력
```

```
df = pd.read_csv('score.csv', nrows=4)
```

```
# 처음 2row 무시, 이후에 4row를 가져옴
```

```
# 상단 타이틀
```

```
df = pd.read_csv('score.csv', skiprows=2, nrows=4)
```

```
print(df)
```

\* 위에서부터 1개 row 삭제후 출력

- 컬럼부터 삭제가 됨.
- 컬럼이 삭제가 되면, row가 컬럼이 되어 버림.
- excel파일 중 row데이터와 상관없는 것을 삭제할때 사용함.

\* column은 존재하고

- 1,3,5번 삭제후 출력

\* column은 존재하고

- Row 중 1,3,5번만 삭제후 출력

\* 지정된 개수만큼 출력

\* pandas\_code.xlsx파일 사용

- \* 1,500번 제외후 10개 데이터 가져옴  
skiprows=[i for i in range(1, 500)],nrows=10)

## 4 파일 저장 및 열기 - txt파일 열기

04\_filesave4.py

```
# 텍스트(.txt)파일 열기
```

```
import pandas as pd
```

```
# txt 파일 열기
```

```
df = pd.read_csv('score.txt',sep='\\t')
```

```
# print(df)
```

```
# index지정해서 열기
```

```
# df = pd.read_csv('score.txt',sep='\\t', index_col='지원번호')
```

```
# print(df)
```

```
df = pd.read_csv('score.txt',sep='\\t')
```

```
df.set_index('지원번호',inplace=True)
```

```
print(df)
```

\* index 지정 후 txt파일열기

\* score.txt파일 열고 난후  
\* index 지원번호 지정

## 4 파일 저장 및 열기 - excel 파일 열기

04\_filesave4.py

```
# 4. 파일저장 및 열기  
# 엑셀(.xlsx)파일 열기  
import pandas as pd
```

```
# 엑셀파일 열기  
df = pd.read_excel('score.xlsx')  
# print(df)
```

```
# index지정해서 열기  
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

```
print(df)
```

\* index 지정 후 excel파일열기

# Quiz

1. 3개의 데이터를 병합하시오
2. 인덱스를 구분 설정하고 데이터를 출력하시오.

```
data1 = pd.read_csv('2014년졸음운전교통사고.csv',encoding='euc-kr')  
data2 = pd.read_csv('2015년졸음운전교통사고.csv',encoding='euc-kr')  
data3 = pd.read_csv('2016년졸음운전교통사고.csv',encoding='euc-kr')
```

1. 3개의 데이터를 병합하시오
2. 인덱스를 구분 설정하고 데이터를 출력하시오.

```
data1 = pd.read_csv('2014년줄음운전교통사고.csv',encoding='euc-kr')  
data2 = pd.read_csv('2015년줄음운전교통사고.csv',encoding='euc-kr')  
data3 = pd.read_csv('2016년줄음운전교통사고.csv',encoding='euc-kr')
```

```
df = pd.concat([data1,data2,data3])  
df.set_index('구분',inplace=True)  
Print(df)
```

# 실습 과제

## 영화

# 5 데이터 확인 - row, column 확인

05\_data1.py

# 5. 데이터 확인 # DataFrame 확인

```
import pandas as pd
```

# 엑셀파일 index지정해서 열기

```
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

# 숫자컬럼인 경우 column별로 개수, 평균, 표준편차, 최소값, 최대값

# 각 지점별 평균, 평균을 보완할 목적으로 사용

# print(df.describe()) # df['키'].describe() 컬럼별

# 컬럼별 타입정보 보여줌

```
# print(df.info())
```

# 처음 5개의 row 데이터를 가져옴.

```
# print(df.head())
```

```
# print(df.head(7)) #처음7개 row 데이터를 가져옴.
```

```
# print(df.tail()) # 마지막 5개 row 데이터를 가져옴
```

```
# print(df.tail(3)) # 마지막 3개 row 데이터를 가져옴.
```

```
print(df.values) # 배열구조로 출력
```

```
print(df.index) # index 출력
```

```
print(df.columns) # 상단의 컬럼 출력
```

```
print(df.shape) # row, column수를 보여줌
```

\* 전체 컬럼의 정보를 아래와 같이 보여

```
df.describe()
```

|       | ram        | hd_size    | screen_size | price         |
|-------|------------|------------|-------------|---------------|
| count | 167.000000 | 167.00000  | 167.000000  | 167.000000    |
| mean  | 6.898204   | 768.91018  | 14.775210   | 64132.898204  |
| std   | 3.787479   | 392.99080  | 1.376526    | 42797.674010  |
| min   | 2.000000   | 32.00000   | 10.100000   | 13872.000000  |
| 25%   | 4.000000   | 500.00000  | 14.000000   | 35457.500000  |
| 50%   | 8.000000   | 1024.00000 | 15.600000   | 47990.000000  |
| 75%   | 8.000000   | 1024.00000 | 15.600000   | 77494.500000  |
| max   | 16.000000  | 2048.00000 | 17.600000   | 226000.000000 |

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 8 entries, 1번 to 8번

Data columns (total 9 columns):

# Column Non-Null Count Dtype

|   |    |            |        |
|---|----|------------|--------|
| 0 | 이름 | 8 non-null | object |
| 1 | 학교 | 8 non-null | object |
| 2 | 키  | 8 non-null | int64  |
| 3 | 국어 | 8 non-null | int64  |

\* 데이터가 너무 많을때 유용함

## 5 데이터 확인 - series 최소값, 최대값..

05\_data2.py

```
# 5. 데이터 확인 - # Series 확인
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')

# series 에 대한 정보 출력
print(df['키'].describe())

# 키의 최소값 - list구조에서 제일 작은 숫자
print("최소값 : ",df['키'].min())

# 키의 최대값
print(df['키'].max())

# 키 큰 순서대로 3명만 가져옴
print(df['키'].nlargest(3))

# 작은 순서대로
print(df['키'].nsmallest(3))
```

```
# 키의 평균
print(df['키'].mean())

# 키의 합
print(df['키'].sum())

# 개수count - 빈공백은 제외
print(df['SW특기'].count())

# 중복을 제거하고 학교이름 출력
print(df['학교'].unique())

# 중복을 제거하고 개수 출력
print(df['학교'].nunique())
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time
import requests
from bs4 import BeautifulSoup

### request 라이브러리 가져오기
# url =
"https://search.daum.net/search?w=tot&q=2023%EB%85%84%EC%98%81%ED%99%94%EC%88%9C%EC%9C%84&DA=M
OR&rtmaxcoll=MOR"
# headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/130.0.0.0 Safari/537.36"}
# res = requests.get(url, headers=headers)
# res.raise_for_status() # 에러시 종료

# selenium 라이브러리 가져오기
for i in range(2020, 2024):
    url =
f"https://search.daum.net/search?w=tot&q={i}%EB%85%84%EC%98%81%ED%99%94%EC%88%9C%EC%9C%84&DA=MOR
&rtmaxcoll=MOR"
    browser = webdriver.Chrome()
    # 이동하려는 주소 입력
    browser.get(url)
    time.sleep(3)
    soup = BeautifulSoup(browser.page_source, "lxml")
    # 파일 저장하기
    with open(f'd1107/screen{i}.html', 'w', encoding='utf-8') as f:
        f.write(soup.prettify())

print("프로그램 완료")
```

# 실습 과제

영화

```
# 웹스크래핑 시작
s_list = []
for year in range(2020,2024):
    with open(f'screen{year}.html','r',encoding='utf-8') as f:
        soup = BeautifulSoup(f,'xml')
        print(f'{year}년도 -----")
        # 기준점
        data = soup.select_one("#mor_history_id_0 > div > div.flipsnap_view > div > div:nth-child(1) > c-flicking-item > c-
layout > div > c-list-doc > ul")
        screens = data.select("li")
        for i,screen in enumerate(screens):
            print(f"{i+1}. -----")
            s_img = screen.select_one(".wrap_thumb img")['src']
            title = screen.select_one(".tit-g.clamp-g").text.strip()
            # number = int(screen.select_one(".conts-desc.clamp-g").text.strip()[3:-2].replace(",",""))
            number = screen.select_one(".conts-desc.clamp-g").text.strip()[3:-2].replace(",","")
            sdate = screen.select_one(".conts-subdesc.clamp-g").text.strip()[:-1]
            # 정보40개 저장됨.
            s_list.append([title,number,sdate])
            # 파일저장을 하지 않고, s_list 모든 데이터를 담아서,
            # 뒤에서 모두 저장

print("리스트 파일 작업완료")
```

# 실습 과제

영화

```
# 파일저장
topTitle = ['제목','관객수','날짜']
with open('screens.csv','w',encoding="utf-8") as f:
    f.write('제목,관객수,날짜\n') # 1번만 글 저장
    for s in s_list:
        f.write(','.join(s)+"\n")

import pandas as pd
df = pd.read_csv('screens.csv',index_col='제목')
Df

dff['관객수'].nlargest()
dff['관객수'].nsmallest()
dff.sort_index(ascending=False)
dff['관객수'].min()
```

# 실습 과제

## 영화

# 6 데이터 선택(기본)-정수index

06\_datachoice1.py

```
# 6. 데이터 선택(기본)
# Column 선택(label) - 정수index로 가져오기
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
# 컬럼으로 가져올때 컬럼을 입력 : df['키'], df['이름']...
# print(df['키'])
# 이름,키 2개를 가져올때
# print(df[['이름','키']])
```

```
# 컬럼 확인 - 정수 index로 가져올때
print(df.columns)
print(df.columns[0]) # 0번째 이름 컬럼확인
print(df.columns[2]) # 2번째 키 컬럼확인
```

```
# 컬럼 : [이름] 가져옴.
print(df[df.columns[0]]) # df['이름'] 와 같음.
# print(df['이름'])
```

```
# 마지막 컬럼을 가져올때
# 데이터의 컬럼수가 너무 많을때 제일 뒤 선택방법
print(df[df.columns[-1]]) # df[df.columns[[-1]]] # [[ 2개 들어감
```

\* 컬럼의 전체 출력  
- df.columns  
\* 첫번째 컬럼의 이름 출력  
- df.columns[0]  
\* 첫번째 컬럼의 내용 출력  
- df['이름']  
- df[df.columns[0]]  
\* 내용 출력은 이름을 입력해야 가능

\* 3번째컬럼 내용 출력  
- df['키']  
- df[df.columns[2]]

\* 컬럼이 너무 많아 옆으로 스크롤하기  
힘들때 마지막 컬럼 출력

- 매일 일자 별 컬럼이 늘어날 때  
- 연령별 인구현황 파일로 실습(뒤에 있음)

# 6 데이터 선택(기본)-정수index: row출력

06\_datachoice2.py

```
# 6. 데이터 선택(기본)
# Column 선택(label) - 슬라이싱
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

```
# 영어컬럼에서 0~4까지 가져옴. 5번 앞까지
print( df['영어'][0:5] )
```

```
# 이름,키 컬럼에서 0-2까지, 3번째 앞까지 가져옴.
print( df[['이름','키']][3] )
```

```
# 3번 앞까지 전체 컬럼을 가져옴.
print(df[:3])
```

```
# [1:] 2번부터, [2:] 3번 다음 이후 다 가져옴
print(df[0:]) #1번째 컬럼부터
print(df[3:]) #4번째 컬럼부터
```

```
print(df.iloc[[0,1,3]]) #row 0,1,3 부분으로 가져올때 사용 , 뒤에 나옴.
```

\* 컬럼도 순번에 들어감  
df[3 : ]

|   | 이름 | 학교     | 키    | 국어  | 영어  | 수학  | 과학  | 사회 | SW특기       |
|---|----|--------|------|-----|-----|-----|-----|----|------------|
| 0 | 1번 | 강나래    | 구로고  | 197 | 90  | 85  | 100 | 95 | Python     |
| 1 | 2번 | 강태원    | 구로고  | 184 | 40  | 35  | 50  | 55 | Java       |
| 2 | 3번 | 강호림    | 구로고  | 168 | 80  | 75  | 70  | 80 | Javascript |
|   | 4번 | 김수찬    | 구로고  | 187 | 40  | 60  | 70  | 75 | NaN        |
|   | 5번 | 김재욱    | 구로고  | 188 | 15  | 20  | 10  | 35 | NaN        |
|   | 6번 | 박동현    | 디지털고 | 202 | 80  | 100 | 95  | 85 | C          |
|   | 7번 | 박해정    | 디지털고 | 188 | 55  | 65  | 45  | 40 | 35         |
|   |    | PYTHON |      |     |     |     |     |    |            |
|   | 8번 | 송근열    | 디지털고 | 190 | 100 | 85  | 90  | 95 | 95         |
|   |    | C#     |      |     |     |     |     |    |            |

\* loc row의 이름으로 가져올 때  
\* iloc row의 번호로 가져올 때

# Quiz

1. 연령별 인구현황 검색 excel파일 저장
2. 상단 3개를 삭제후 가져오기
3. 행정기관 index 지정
4. 상단 3개만 출력하시오.

연령별 인구현황 검색 : Excel파일 다운로드

[ 설정 ]

전국 월간 2012년 02월~2012년 02월

계 제외, 연령구분단위 5세, 연령구분 0-100세이상

<https://jumin.mois.go.kr/ageStatMonth.do>

연령별 인구현황

통계표

그래프

행정구역: 전국 ▼ 시·군·구 ▼ ⓘ

등록구분: 전체 ▼ ⓘ

조회기간: ☒ 월간 ☐ 연간 2012년 ▼ 02월 ▼ - 2012년 ▼ 02월 ▼ ⓘ

※ 매월 말일 작성 / 공표일시 : 매월 1일 12시 이후(공표일이 주말·공휴일인 경우에는 다음 평일에 공표)

구분: ☐ 계 ☒ 남·여 구분

정렬순서: 행정기관코드 ▼ 오름차순 ▼

연령 구분 단위: 5세 ▼

만 연령구분: 0 ▼ 100이상 ▼

검색 🔍

초기화 ↺

● 현재화면 ○ 전체시군구현황 ○ 전체읍면동현황

📄 csv 파일 다운로드

📄 excel 파일 다운로드

```
import pandas as pd
```

```
# 남자데이터 출력
```

```
df = pd.read_excel('201202_201202_연령별인구현황_월간.xlsx', skiprows=3, index_col='행정기관', usecols='B,E:Y')
print(df.head(3))
```

```
# 1,195,951 -> 1195951 숫자로 변경(정수형으로 변경)
```

```
df.iloc[0] = df.iloc[0].str.replace(',', '').astype(int)
```

```
# 전국데이터 출력
```

```
print(df.iloc[0])
```

```
# 여자데이터 출력
```

```
df_w = pd.read_excel('201202_201202_연령별인구현황_월간.xlsx', skiprows=3, index_col='행정기관', usecols='B,AB:AV')
print(df_w.head(2))
```

```
# 0~4세, 1.1이라는 것이 붙음. 2개가 있기에 붙여짐.
```

```
print(df.columns)
```

```
print(df_w.columns) # 1.1이라는 것 붙음
```

```
df_w.columns = df.columns # 컬럼명을 남자의 것으로 통일
```

```
print(df_w.columns)
```

```
# 1,195,951 -> 1195951 숫자로 변경(정수형으로 변경)
```

```
df_w.iloc[0] = df_w.iloc[0].str.replace(',', '').astype(int)
```

```
print(df_w)
```

## 연령별 인구현황

\* skiprows=3

- 위 3줄을 삭제 후 가져옴.

\* index\_col='행정기관'

- Index를 행정기관으로 설정

\* usecols='B,E:Y'

- 사용할 컬럼은  
B컬럼과 E에서 Y까지의 컬럼사  
용

\* , 심표를 없애고 int형으로 변경

- replace(',', '', '')

- astype(int)

```
df= pd.read_csv('score.csv', sep = '\t', thousands = ',')
```

```
# 천단위 제거후 가져오기
```

```
-----
```

```
# 오라클 sql구문 : 천단위 제거후 가져오기
```

```
SELECT replace(column_name, ',', '')::numeric FROM table_name
```

연령별  
인구현황

# Quiz

1. 출생아수 합계출산율 검색 > excel파일 저장
2. 상단 2개를 삭제후 가져오기
3. 0번 컬럼을 index 지정 , index이름 : 목록으로 지정
4. 3개만 출력하시오. 5. 출생아 수 index를 출력하시오.

출생아수 합계출산율 검색 : 엑셀버튼 클릭 > 엑셀파일 저장 | 그래프 canvas javascript  
[https://www.index.go.kr/potal/main/EachDtlPageDetail.do?idx\\_cd=1428](https://www.index.go.kr/potal/main/EachDtlPageDetail.do?idx_cd=1428)

[단위 : 천명, 가임여자 1명당 명]

|        | 2012  | 2013  | 2014  | 2015  | 2016  | 2017  | 2018  | 2019  | 2020  |
|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
|        | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   | ▲▼□   |
| 출생아 수  | 484.6 | 436.5 | 435.4 | 438.4 | 406.2 | 357.8 | 326.8 | 302.7 | 272.3 |
| 합계 출산율 | 1.297 | 1.187 | 1.205 | 1.239 | 1.172 | 1.052 | 0.977 | 0.918 | 0.840 |

```
import pandas as pd
```

```
# 상단 2개 제외, 처음1개는 컬럼으로 지정되어서, 2개만 가져오면 됨.
# index.name이 없으면 번호로 넣으면 됨. 0
Df = pd.read_excel('stat_142801.xls',skiprows=2,nrows=2,index_col=0)
Print(df) # df.index.name = '목록'
```

```
# 출생아 수 0번째 row를 출력하려면 에러가 남.
# print(df.loc[0])
# index 출력 (출생아 수 -> 출생아\xa0수 되어 있음)
print(df.index) # index 잘 나오지만 df.index.values 출력하면 다름
```

```
# index list형태로 모두 출력
print(df.index.values)
# 이렇게 출력됨 ['출생아\xa0수' '합계\xa0출산율'] # 유니코드 띄워쓰기가 들어감
```

```
# index name변경을 해줌.
df.rename(index={'출생아\xa0수':'출생아 수','합계\xa0출산율':'합계 출산율'},inplace=True)
print(df.loc['출생아 수'])
```

```
df = df.T # row,column축 변경
print(df)
```

## 합계 출산율

\* skiprows=2  
- 제일 row는 컬럼으로 설정 됨.

\* index\_col='지원번호'  
index.name이 없으면  
index 번호를 넣으면 됨.  
( 0,1,2,3....순으로 진행됨. )

\* Index에 특수문자가 들어가  
있어 에러가 남.

\* df.index.values를 해야 보임.  
[ 출력 형태 ]  
출생아\xa0수' '합계\xa0출산율

\* index이름 변경  
df.index.values[3]='부산광역시'

\* 축을 변경할 수 있음.

## 7 데이터 선택(loc)-이름을 이용, row에서 원하는 col선택

07\_dataloc1.py

```
# 7. 데이터 선택(loc)
# 이름을 이용하여 원하는 row에서 원하는 col선택
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

```
print(df['이름']) # 이름컬럼 전체 출력
# index 1번에 해당하는 전체 row데이터
# print(df.loc['1번'])
# print(df.loc['5번'])
```

```
# index 1번 학생, 국어컬럼만 가져옴
print(df.loc['1번','국어']) # 90나타남.
```

```
print(df.loc[['1번','3번'],'영어']) #1번,3번 학생 영어성적 컬럼만
print(df.loc[['1번','3번'],['영어','수학']]) #1번,3번 학생 영어,수학 성적 컬럼만
print(df.loc[['1번','3번']]) #1번,3번 전체 컬럼 출력
```

```
# 1번~5번까지 학생, 국어~사회성적까지 출력
# 보통 : 하면 앞전까지 인데... 여기는 5번 포함 됨
print(df.loc['1번':'5번','국어':'사회'])
```

\* ['1번':'국어']  
- [ row, column ] - row.column 넣음

\* ['1번':'5번']  
[ Index이름:index이름, 컬럼이름:컬럼이름 ]

이름으로 가져오기에 1번에서 5번 포함 가져옴.

## 8 데이터 선택(iloc)-번호를 이용, row에서 원하는 col선택

08\_datailoc1.py

```
# 8. 데이터 선택(iloc)
# 위치를 이용하여 원하는 row에서 원하는 col선택
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)
# 0번, 1번째 위치의 데이터 출력
# print(df.iloc[0])
# print(df.iloc[4]) # 4번, 5번째 학생 출력
print(df.iloc[0:5]) # 0번부터 4번까지, 5번앞까지 출력

# 0번, 1컬럼 (0,1) - 1번째, 학교(1)가 출력
# print(df.iloc[0,1]) # 구로고 출력
# 4번,2컬럼 (4,2) - 5번째,키 가 출력
print(df.iloc[4,2]) # 188 출력
print(df.iloc[[0,1],2]) # 0,1번 row 키 출력
print(df.iloc[[0,1],[3,4]]) # 0,1번 row 국어,영어 출력

# row 0-4번까지-5번앞까지, column 3-7번까지 출력
# 국어-사회까지
print(df.iloc[0:5,3:8])
```

\* [0:5,3:8]  
0,1,2,3,4 - 5개 row를 가져옴.  
3,4,5,6,7 - 5개 column을 가져옴.

## 9 데이터 선택(조건)-if

09\_dataif1.py

```
# 9. 데이터 선택(조건)
# 조건에 해당하는 데이터 선택
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 키가 185 이상인지 여부 True,False 출력
print(df['키']>=185)

# 필터조건을 변수로 지정
filt = (df['키']>=185)
# print(df[filt]) #필터 조건으로 출력
# print(df[df['키']>=185]) # 변수를 바로 적용
# print(df[~filt]) # 필터 조건 반대로 출력

# 키가 185 이상인 학생들의 수학데이터 출력
# df.loc[row,column] -> [ df['키']>=185 , '수학' ]
print(df.loc[df['키']>=185,'수학' ])
print(df.loc[df['키']>=185,['이름','수학','과학' ]])
```

\* 변수에 조건을 저장  
filt = (df['키']>=185)

df[filt] -> 조건 변수를 통해 검색

\* df.loc [ row , column ]  
- Row 자리 : df['키']>=185  
- Column 자리 : 수학  
[ df['키']>=185 , '수학' ]

## 9 데이터 선택(조건)-if and, or조건

09\_dataif2.py

```
# 9. 데이터 선택(조건)
# 조건문 & and 그리고
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)
```

```
# 키가 185보다 크고, 구로고에 다니는 학생 검색
print( df.loc[(df['키']>=185) & (df['학교']=='구로고')] )
```

```
# | or 또는 조건
# 키가 170보다 작거나, 200보다 큰 학생 검색
print( df.loc[(df['키']<170) | (df['키']>200) ] )
```

\* and, or 조건 검색

```
*
fdata = df['영어']>60
print(df[fdata][['영어','수학']].count())
```

## 9 데이터 선택(조건)-str 함수 1

09\_dataif3.py

```
# 9. 데이터 선택(조건)
# str함수
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 이름이 송으로 시작하는 학생 검색
filt = df['이름'].str.startswith('김')
print(df[filt])

# 이름이 근이 들어가는 학생 검색
filt = df['이름'].str.contains('근')
print(df[filt])
print(df[~filt]) # 근이 들어가는 학생 제외 학생출력
```

## 9 데이터 선택(조건)-str 함수 2

09\_dataif3.py

```
# 특기가 python,java 인 학생 검색 - ## -> 대소문자 구분을 함.  
# PYTHON 대문자 검색이 안됨.  
langs = ['python','java']  
filt = df['SW특기'].isin(langs)  
print(df[filt])
```

```
# 검색할 문자를 모두 소문자로 변경 lower()  
langs = ['python','java']  
filt = df['SW특기'].str.lower().isin(langs)  
print(df[filt])
```

```
# sw특기 중 Nan이 있으면 error가 남.  
filt = df['SW특기'].str.contains('Java')  
# print(df[filt])
```

```
print(df['SW특기'].str.lower())
```

```
# 검색할 문자를 모두 소문자로 변경 lower()  
langs = ['python','java']  
filt = df['SW특기'].str.lower().isin(langs)  
print(df[filt])
```

## 9 데이터 선택(조건)-str 함수 3

09\_dataif3.py

```
# isin(langs) 검색시 Nan은 False로 검색이 됨
# contains 검색시 Nan은 Nan 검색이 되어 조건 실패됨.
print(df['SW특기'].str.lower().isin(langs)) # Nan->False
## Nan -> Nan 출력
print(df['SW특기'].str.contains('Java'))
## Nan -> True,False 출력
print(df['SW특기'].str.contains('Java',na=True))
print(df['SW특기'].str.contains('Java',na=False))
## 문자열 함수 2번사용, str,str 2번
filt = df['SW특기'].str.lower().str.contains('java',na=True)
print(df[filt])

# Java가 들어가는 SW특기 검색
filt = df['SW특기'].str.contains('Java', na=False)
print(df[filt])
```

\* Nan 부분이 error 남

\* 대소문자 구분없이 검색  
\* case=True 대소문자 구분 검색  
namedata = df['이름'].str.contains('to',  
case=False)

# 10 결측치 - 데이터 채우기 fillna()

10\_datafill1.py

```
# 10. 결측치
# # 데이터 채우기 fillna
import pandas as pd    결측값이면 True라고 표시
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)
```

```
# Nan 데이터를 빈 칸으로 채움
print(df.fillna(""))
print(df.fillna('없음'))
```

```
# 학교 데이터 전체를 Nan으로 채움
import numpy as np
df['학교'] = np.nan
print(df)
```

```
# Nan 으로 되어 있는 모든 칸을 모름으로 채움
# inplace=True 해야 반영됨.
df.fillna('모름',inplace=True)
# 반영하지 않으면 변경되지 않음.
print(df)
```

| 이름 | 학교  | 키   | 국어  | 영어  | 수학  | 과학  | 사회 | SW특기 |            |
|----|-----|-----|-----|-----|-----|-----|----|------|------------|
| 1번 | 강나라 | NaN | 197 | 90  | 85  | 100 | 95 | 85   | Python     |
| 2번 | 강태원 | NaN | 184 | 40  | 35  | 50  | 55 | 25   | Java       |
| 3번 | 강호림 | NaN | 168 | 80  | 75  | 70  | 80 | 75   | Javascript |
| 4번 | 김수찬 | NaN | 187 | 40  | 60  | 70  | 75 | 80   | NaN        |
| 5번 | 김재욱 | NaN | 188 | 15  | 20  | 10  | 35 | 10   | NaN        |
| 6번 | 박동현 | NaN | 202 | 80  | 100 | 95  | 85 | 80   | C          |
| 7번 | 박해정 | NaN | 188 | 55  | 65  | 45  | 40 | 35   | PYTHON     |
| 8번 | 송근영 | NaN | 190 | 100 | 85  | 90  | 95 | 95   | C#         |

.isna(): 결측값이면 True라고 표시  
.isnull(): 같은 명령어

\* inplace=True 적용해야 반영이 됨.

# 10 결측치 - 데이터 채우기 fillna()

10\_datafill2.py

```
# 10. 결측치  
# # 데이터 채우기 fillna  
import pandas as pd  
# 엑셀파일 index지정해서 열기  
df = pd.read_excel('score.xlsx', index_col='지원번호')  
print(df)
```

```
# SW특기 데이터 중에서 Nan에 대해서 채움  
# 반영하려면 inplace=True  
df['SW특기'].fillna('확인 중',inplace=True)  
print(df)
```

\* 일부 컬럼만 Nan 채울수 있음

# 10 결측치 - 데이터 삭제 dropna() 1

10\_datafill3.py

```
# 10. 결측치  
# # 데이터 삭제, dropna()  
import pandas as pd  
# 엑셀파일 index지정해서 열기  
df = pd.read_excel('score.xlsx', index_col='지원번호')  
print(df)
```

```
# Nan가 포함되어 있는 row 전체를 삭제  
# inplace=True 해야만 반영이 됨.  
# print(df.dropna())  
print(df.dropna(inplace=True))
```

\* 일부 컬럼만 Nan 채울수 있음

# 10 결측치 - 데이터 삭제(dropna()) 3

10\_datafill4.py

```
# 10. 결측치
# # 데이터 삭제, dropna()
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)
```

```
# row에 Nan이 1개라도 있으면 삭제
# inplace=True 해야 반영됨.
print(df.dropna(axis='index', how='any'))
# print(df) 해보면 반영이 안되어 있음.
```

```
# SW특기 컬럼이 삭제
print(df.dropna(axis='columns'))
```

```
# 학교 컬럼 모두 Nan 입력
import numpy as np
df['학교'] = np.nan
```

```
# 컬럼이 모두 Nan일때 삭제 - 학교컬럼 삭제
print(df.dropna(axis='columns', how='all'))
```

[ df.dropna 세부 설명 ]

axis : index or columns

index

- row를 삭제, columns - 컬럼을 삭제

how : any or all

- any - Nan이 1개라도 있으면 삭제,

- all - 모두 있어야 삭제

# 11 데이터 정렬 - sort\_values 오름차순, 내림차순 1

11\_datasort1.py

```
# 11. 데이터 정렬
# # sort_values 오름차순, 내림차순
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 키를 기준으로 오름차순(낮은키 부터) 정렬
print(df.sort_values('키'))

# 키를 기준으로 내림차순 정렬
print(df.sort_values('키',ascending=False))

# 수학,영어를 기준으로 내림차순 정렬
# 3번, 4번 수학 70, 영어 75,60 로 정렬됨.
print(df.sort_values(['수학','영어'],ascending=False ))
# 수학, 영어 점수 기준으로 오름차순 정렬
# 3번, 4번 70, 영어 60,75로 정렬 됨.
print( df.sort_values(['수학','영어']))
# print( df.sort_values(['수학','영어'],ascending=False))

# 수학 오름차순, 영어-내림차순 정렬
print(df.sort_values(['수학','영어'],ascending=[True,False]) )
```

[ 정렬 ]  
ascending=True : 오름차순  
ascending=False : 내림차순

# 11 데이터 정렬 - sort\_values 오름차순, 내림차순 2

11\_datasort1.py

```
# series 1차원도 가능 - 키 기준으로 오름차순 정렬  
print(df['키'].sort_values() )
```

```
# 키 기준 내림차순 정렬  
print(df['키'].sort_values(ascending=False) )
```

```
# 지원번호 기준 오름차순  
print( df.sort_index() )
```

```
# 지원번호 기준으로 내림차순  
print( df.sort_index(ascending=False) )
```

[ 정렬 ]  
index기준으로 정렬

# 12 데이터 수정 - column 수정

12\_dataupdate1.py

```
# 12. 데이터 수정
# # column 수정
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)

# 학교 컬럼중에 구로고를 영등포고로 변경
# print(df['학교'].replace({'구로고':'영등포고'}))
# 2개 이상일때
print(df['학교'].replace({'구로고':'영등포고','디지털고':'구청고'}))
```

```
# inplace=True 해야 반영이 됨.
df['학교'].replace({'구로고':'영등포고'},inplace=True)
print(df)
```

```
# SW특기 컬럼 내용을 모두 소문자 변경
print(df['SW특기'].str.lower())
# SW특기 컬럼 - 대문자 변경
print(df['SW특기'].str.upper())
# 소문자 변경후 저장
df['SW특기'] = df['SW특기'].str.lower()
print(df)
```

```
# 고등학교를 입력하는 방법
df['학교'] = df['학교'] + '등학교'
print(df)
```

Inplace=True 해야 반영이 됨.

# 12 데이터 수정 - column 추가

12\_dataupdate2.py

```
# 12. 데이터 수정
# # column 추가
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)

#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)

# 결과 column을 추가하고 전체 데이터는 Fail 초기화
df['결과'] = 'Fail'
print(df)

# 총합이 400보다 큰 데이터에 대해서 합격
df.loc[df['총합'] > 400, '결과'] = 'Pass'
print(df)
```

컬럼 추가

컬럼 값 비교해서 수정

# 12 데이터 수정 - drop(column) 삭제

12\_dataupdate3.py

```
# 12. 데이터 수정
# # column 삭제
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)

#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)

# 총합 column 삭제, inplace=True해야 반영
df.drop(columns=['총합'],inplace=True)
print(df)

# 국어,영어,수학 column 삭제
df.drop(columns=['국어','영어','수학'],inplace=True)
print(df)
```

Inplace=True 해야 반영됨.

여러 컬럼 삭제

# 12 데이터 수정 - drop(index): row 삭제

12\_dataupdate4.py

```
# 12. 데이터 수정
## row 삭제
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)
```

```
# 4번 index, row 삭제, inplace=True해야 반영됨.
# df.drop(index='4번',inplace=True)
# print(df)
```

```
# 수학적수가 80점 미만 학생만 남기고, 삭제
# 1번=100,6번=95,8번=90 이상인 학생 삭제
filt = df['수학'] < 80
# print(df[filt])
```

```
# 80점 밑으로 삭제시
# index를 출력하면 80점 밑으로 index가 출력됨.
print(df[filt].index)
# 이 index번호를 drop에 넣어주면 됨.
print(df.drop(index=df[filt].index))
# inplace=True 해야 반영됨.
print(df)
```

```
#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)
```

```
# 결과 column을 추가하고 전체 데이터는 Fail 초기화
df['결과'] = 'Fail'
# print(df)
```

```
# 총합이 400보다 큰 데이터에 대해서 합격
df.loc[df['총합'] > 400,'결과'] = 'Pass'
# print(df)
```

80점 미만 조건을 넣으면  
80점 미만 남기고, 모두 삭제 됨.

80점 미만만 삭제하고 싶으면  
- filter index를 넣어야 함.

df.drop(index=['1번','3번']) # index를 가지고 삭제

# 12 데이터 수정 - row 추가, 일부변경, 삭제, cell 수정

12\_dataupdate5.py

```
# 12. 데이터 수정
## row 추가, cell수정
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)
```

```
# 새로운 row추가
df.loc['9번'] = ['유승찬','단지고',184,90,90,90,90,90,'Django',(90*5),'Pass']
print(df)
```

```
# cell 수정 - 4번학생, SW특기 수정
df.loc['4번','SW특기'] = 'Python'
print(df)
```

```
# cell 5번 학생의 학교-디지털고, SW특기-C 수정
df.loc['5번',['학교','SW특기']] = ['디지털고','C']
print(df)
```

```
#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)
```

```
# 결과 column을 추가하고 전체 데이터는 Fail 초기화
df['결과'] = 'Fail'
# print(df)
```

```
# 총합이 400보다 큰 데이터에 대해서 합격
df.loc[df['총합'] > 400,'결과'] = 'Pass'
# print(df)
```

새로운 row추가  
- 숫자는 +, \* 계산이 됨.

Cell 수정  
- 해당 cell수정

# 12 데이터 수정 - column 순서 변경

12\_dataupdate6.py

```
# 12. 데이터 수정
## column 순서 변경
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)
```

```
#column순서 변경
cols = list(df.columns)
print(cols)
```

```
# 맨뒤 결과 column을 앞으로 변경, 나머지 column 순차적으로 출력
# cols[-1] -> list여야 해서, [ ]안에 추가적으로 넣음.
```

```
# 1개변수는 리스트로 변경해야 더하기가 가능함
df = df[ [cols[-1]] + cols[0:-1]]
# df = df[['결과','이름','학교']] #결과, 이름, 학교 출력 됨.
print(df)
```

```
#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)
```

```
# 결과 column을 추가하고 전체 데이터는 Fail 초기화
df['결과'] = 'Fail'
# print(df)
```

```
# 총합이 400보다 큰 데이터에 대해서 합격
df.loc[df['총합'] > 400, '결과'] = 'Pass'
# print(df)
```

```
** list 타입이어야 list + list 가능
컬럼 제일 뒤에 있는 것을
제일 앞으로 출력하고
다음은 순차적으로 출력
```

# 12 데이터 수정 - column 이름 변경

12\_dataupdate7.py

```
# 12. 데이터 수정
## column 이름 변경
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
# print(df)
```

```
# 컬럼column 이름 변경
print(df.columns)
# 전체컬럼 이름 변경
```

```
#df.columns=['name','school','height','kor','eng','math','science','society','sw','total',
',result']
```

```
df.rename(columns={'이름':'name','학교':'school'},inplace=True)
print(df)
```

```
#column추가
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
print(df)
```

```
# 결과 column을 추가하고 전체 데이터는 Fail 초기화
df['결과'] = 'Fail'
# print(df)
```

```
# 총합이 400보다 큰 데이터에 대해서 합격
df.loc[df['총합'] > 400, '결과'] = 'Pass'
# print(df)
```

(전체, 부분) 컬럼 이름을 변경 방법

```
# 전체 열 이름 입력하기
# df.columns = ['col', 'col', 'col']
```

```
# 선택하여 열 이름 변경하기
# df.rename(columns={'Before':'After'})
```

# 13 함수적용 - column 내용 변경

13\_datafunction1.py

```
# 13. 함수적용
## 컬럼 내용 변경
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 학교 글자에 문자를 추가하는 것은 가능 ( 문자형데이터 + 문자형데이터 )
# 숫자에 글자를 추가하는 것은 불가능 - 함수를 적용해서 처리
df['학교'] = df['학교']+'등학교'
print(df)

# 키에 cm을 추가하려면 에러 (예: 197cm)
# 키 정수형데이터 + 문자형데이터 = error
# df['키'] = df['키']+'cm'
# print(df)

# 정수형데이터 + 정수형데이터 가능
df['키'] = df['키'] + 100
print(df)
```

문자형데이터 + 문자형데이터  
( 가능 )

정수형데이터 + 문자형데이터  
( 불가능 )

정수형데이터 + 정수형데이터  
( 가능 )

# 13 함수적용 - column 내용 변경

13\_datafunction1.py

```
# 13. 함수적용
## 컬럼 내용 변경
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)
```

```
# 키 뒤에 cm을 추가하는 함수
def add_cm(height):
    return str(height)+'cm'
```

```
# 키 데이터를 add_cm함수를 호출해서 적용후 리턴
df['키'] = df['키'].apply(add_cm)
print(df)
```

```
# SW특기 첫글자 대문자 뒤 소문자로 변경
def capchange(lang):
    if pd.notnull(lang): # Nan인지 확인
        return lang.capitalize() #첫글자 대문자, 나머지 소문자
    return lang
```

```
# capchange함수적용
# df['SW특기'] = df['SW특기'].apply(capchange)
# print(df)
```

```
# 함수 적용하지 않아도 첫글자 대문자, 나머지 소문자적용
print(df['SW특기'].str.capitalize())
```

키 뒤에 cm추가  
함수사용

첫글자 대문자, 나머지 소문자  
함수사용

df['SW특기'] = df['SW특기']  
.str.capitalize()

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| upper      | 주어진 문자열에서 모든 알파벳들을 대문자로 변환시킨다.                                            |
| capitalize | 주어진 문자열에서 맨 첫 글자를 대문자로 변환시킨다.                                             |
| title      | 주어진 문자열에서 알파벳 외의 문자(숫자, 특수기호, 띄어쓰기 등)로 나누어져 있는 영단어들의 첫 글자를 모두 대문자로 변환시킨다. |

```
##### 문자열 함수 : 문자만 가능 #####
# str[:] : 문자슬라이싱
# str.split() : 분할 str.split(",")
# str.startswith() :
시작글자 인식 str.startswith("서울")
# str.endswith() :
끝글자 인식 str.endswith(".py")
# str.contains() : 포함글자 인식 str.contains('s',na=False)
# str.find() : 문자위치찾기 str.find('안')
# str.replace() : 문자 대체 str.replace(",":"")
# str.pad() : 문자열 패딩,
특정글자로 채워줌 str.pad(width=20,side='left',fillchar='_')
# str.zfill() : 0으로 채워줌 str.zfill(width=10)
df['국어'].astype(str).str.zfill(width=3)

# str.strip() : 공백제거
# str.lower() : 소문자 변경
# str.upper() : 대문자 변경
# str.swapcase() : 대소문자 변경
```

```
# 배열을 함수 적용하려면 apply 사용
# 개별 int,float,str 단일타입을 사용하려면 함수명(값) 호출
def h_func(x):
    return int(x/2)

plt.bar(df['이름'],df['국어'])
## 데이터가 단일타입 - int,float,str
for i in range(8):
    plt.text(df['이름'][i],h_func(df['국어'][i]),df['국어'][i],ha='center')
```

# Quiz

1. 요일컬럼 추가
2. 0-6까지 숫자를 랜덤으로 입력
3. 함수생성 : 0-월, 1-화, 2-수.....6-일 입력
4. 요일 컬럼을 키 바로 뒤로 컬럼순서를 변경하시오.

| 이름    | 학교   | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 요일 |
|-------|------|-----|-----|-----|-----|----|----|------------|----|
| 0 강나래 | 구로고  | 197 | 90  | 85  | 100 | 95 | 85 | Python     | 수  |
| 1 강태원 | 구로고  | 184 | 40  | 35  | 50  | 55 | 25 | Java       | 토  |
| 2 강호림 | 구로고  | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | 토  |
| 3 김수찬 | 구로고  | 187 | 40  | 60  | 70  | 75 | 80 |            | 토  |
| 4 김재욱 | 구로고  | 188 | 15  | 20  | 10  | 35 | 10 |            | 목  |
| 5 박동현 | 디지털고 | 202 | 80  | 100 | 95  | 85 | 80 | C          | 금  |
| 6 박혜정 | 디지털고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     | 일  |
| 7 송근열 | 디지털고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | 일  |

# 13 Quiz - 함수

.py

```
# 기본 데이터
import random
df['요일'] = 0      # 최초 1로 세팅
for i in range(8):
    df.loc[i,'요일'] = random.randint(0,6)    #각 요소에 랜덤 0,6까지 숫자 입력
```

# 함수 입력

```
# 함수호출 - 요일 글자로 변경해서 수정
df['요일'] = df['요일'].apply(change)

print(df)
```

```
def change(x):
    if x == 0 :
        return '월'
    elif x == 1 :
        return '화'
    elif x == 2 :
        return '수'
    elif x == 3 :
        return '목'
    elif x == 4 :
        return '금'
    elif x == 5 :
        return '토'
    else :
        return '일'
```

실습  
과제

# Quiz

1. 금액컬럼을 추가 하시오. (금액: 달러 + 달러\*우대\*0.01 )
2. 우대가 50 이상인 사람에게만 달러 + 달러\*우대\*0.01  
50미만은 달러만 입력

```
data = {  
    '번호': ['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'],  
    '이름': ['홍길동', '유관순', '이순신', '강감찬', '김구', '김유신', '홍길자', '홍길순'],  
    '달러': ['$19,700', '$18,400', '$16,800', '$18,700', '$18,800', '$20,200', '$18,800', '$19,000'],  
    '우대': [90, 40, 80, 40, 15, 80, 55, 100]  
}
```

# 13 Quiz - 함수

.py

# 필터 적용방법

```
filt = df['우대'] >= 50
```

```
df.loc[filt, '금액'] = df['달러'] + df['달러'] * df['우대'] * 0.01
```

```
df.loc[~filt, '금액'] = df['달러']
```

# 함수 적용방법

```
def change(data):
```

```
    if data['우대'] >= 50: return data['달러'] + (data['달러'] * data['우대'] * 0.01)
```

```
    else : return data['달러']
```

```
df.loc[filt, '금액'] = df.apply(change, axis=1)
```

```
df
```

axis=1이면 각 행(row) 이 함수에 전달  
axis=0이면 각 열(column) 이 함수에 전달

실습  
과제

# 14 그룹화-통계, 평균

14\_datagroup1.py

```
# 14. 그룹화
## 동일한 값을 가진 것들을 그룹으로 해서 통계,평균 등 값을 계산
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)
# 그룹 설정
print(df.groupby('학교'))
```

```
# 그룹해서 구로고 정보 가져옴.
print( df.groupby('학교').get_group('구로고') )
# 그룹해서 디지털고 정보 가져옴.
# print( df.groupby('학교').get_group('디지털고'))
```

```
# 학교별 계산가능한 컬럼 평균 구함.
print(df.groupby('학교').mean())
# 그룹 학교별 크기(수)가 구해짐.
print(df.groupby('학교').size())
```

```
# 그룹 학교별 구로고 크기만 구해짐 - 5명
print(df.groupby('학교').size()['구로고'])
```

```
# 학교로 그룹화 후 키 평균을 구함.
print(df.groupby('학교')['키'].mean())
```

```
# 학교로 그룹화 하여 키, 국어, 영어, 수학 평균을 구함.
print(df.groupby('학교')[['키','국어','영어','수학']].mean())
```

그룹해서 구로고 정보 가져옴.

학교 그룹으로 여러 컬럼 보기

### 그룹화

# 컬럼을 기준으로 그룹핑 - df.groupby(컬럼) : sum():합계,count():개수,size():크기

df.groupby('학교').count()

df.groupby(['학교','학년']).count()

df['학교'].value\_counts()

# 컬럼지정필요 : df.groupby(컬럼)[숫자타입컬럼] mean(),max(),min()

df.groupby('학교')['키'].min()

df.groupby('학교')[['키','국어']].min()

### 그룹별 키로 정렬 , ascending=False : 역순정렬

df.groupby('학년')[' '].mean().sort\_values(ascending=False)

### value\_count() : 그룹핑 개수

df.groupby('학교')['학년'].value\_counts().loc['구로고']

df.groupby('학교')['학 년'].value\_counts(normalize=True).loc['구로고']

# 14 그룹화 - 평균 정렬

14\_datagroup2.py

```
# 14. 그룹화
## 동일한 값을 가진 것들을 그룹으로 해서 통계, 평균 등 값을 계산
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 학년 컬럼 추가
df['학년']=[3,3,2,1,1,3,2,2]
print(df)

# 학교,학년별 그룹화 평균구함
print(df.groupby(['학교','학년']).mean())

# 학년별 그룹화 평균구함.
print(df.groupby("학년").mean())

# 학년별 그룹화 키폭균 순차정렬
print(df.groupby("학년").mean().sort_values('키'))

# 학년별 그룹화 키폭균 역순정렬
print(df.groupby("학년").mean().sort_values('키',ascending=False))
```

그룹해서 정렬

```
# 14. 그룹화
## 동일한 값을 가진 것들을 그룹으로 해서 통계,평균 등 값을 계산
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 학년 컬럼 추가
df['학년']=[3,3,2,1,1,3,2,2]
print(df)

# 학년별 그룹화 키합계
print(df.groupby(['학교','학년']).sum())
# print(df.groupby('학년').sum().sort_values('키'))

# 학교로 그룹화를 한뒤 sw특기로 개수
# Nan을 제외한 개수 구르고3명, 디지텔고 3명
print(df.groupby('학교')['SW특기'].count())

# 학교로 그룹화해서 이름, sw특기로 개수 확인
print( df.groupby('학교')[['이름','SW특기']].count() )
```

```
# 14. 그룹화
## 동일한 값을 가진 것들을 그룹으로 해서 통계, 평균 등 값을 계산
import pandas as pd
# 엑셀파일 index지정해서 열기
df = pd.read_excel('score.xlsx', index_col='지원번호')
print(df)

# 학년 컬럼 추가
df['학년']=[3,3,2,1,1,3,2,2]
print(df)

# 학교로 그룹화 후 학년으로 학생수 확인
# df.groupby('학교')['학년'].value_counts() 2줄로 표현
school = df.groupby('학교')
print(school['학년'].value_counts())

# 학교로 그룹화 후 구로고 학생 학년으로 학생수 확인
# 구로고 1학년 2, 3학년-2명, 2학년-1명
print(school['학년'].value_counts().loc['구로고'])
# print(school['학년'].value_counts().loc['디지털고'])

# 퍼센트로 확인(normalize=True)
print( school['학년'].value_counts(normalize=True).loc['구로고'] )
```

# 14 그룹화 - agg 함수

14\_datagroup4.py

```
# 학교,성별로 국어,영어,수학점수 평균을 출력하시오.
df.groupby(['school','gender'])[['kor','eng','math']].mean()['kor']
# df.groupby(['school','gender'])[['kor','eng','math']].mean().loc['디지 털고']['kor'].loc['Female']
# df.groupby(['school','gender'])[['kor','eng','math']].mean().loc['디지 털고']
# df.groupby(['school','gender'])[['kor','eng','math']].mean().sort_values('kor')
# df.groupby(['school','gender'])[['kor','eng','math']].mean().sort_values('kor')

# df.groupby('grade')['total'].mean()
# df.groupby('grade')['total'].min()
# df.groupby('grade')['total'].max()
# df.groupby('grade')['total'].sum()

# apply함수 - 함수호출, agg - 기본함수호출
df.groupby(['school','grade'])['total'].agg(['mean','min','max','sum'])
```

# 15 DataFrame 합치기 - merge, concat

```
import pandas as pd

# DataFrame Join
# concat 행기준으로 - 각각의 DataFrame 합침
# df = pd.concat([data1,data2,data3])

# 2차원데이터
data ={
    '이름':['주바다','공유진','송선유','양홍욱'],
    '키':[180,182,188,179],
    '몸무게':[47,49,50,45]
}

data2={
    '이름':['주바다','공유진','송선유','윤상운'],
    '전공':['컴퓨터공학','매카닉공학','독일어학','컴퓨터공학'],
    '실력':['고급','중고급','특급','중고급']
}
```

```
df = pd.DataFrame(data)
df2 = pd.DataFrame(data2)
```

```
#### DataFrame join하는 방법
#### merge - 열기준(이름)으로 합침
# inner - data,data2 같은 데이터가 있는 경우만 합침
# df = 주바다,공유진,송선유,양홍욱
# df2 = 주바다,공유진,송선유,윤상운
```

# 15 DataFrame 합치기 - merge, concat

```
import pandas as pd
```

```
# DataFrame Join
```

```
# concat 행기준으로 - 각각의 DataFrame 합침
```

```
# df = pd.concat([data1,data2,data3])
```

2개의 배열 생성

```
df = pd.DataFrame(data)
```

```
df2 = pd.DataFrame(data2)
```

```
# inner - data, data2 같은 데이터가 있는 경우만 합침
```

```
mdf = pd.merge(left=df, right=df2, how='inner', on='이름')
```

```
print(mdf)
```

```
# left=df 기준으로 합침
```

```
mdf2 = pd.merge(left=df, right=df2, how='left', on='이름')
```

```
# mdf2['전공'][3]='자율전공'
```

```
# mdf2['실력'][3]='초급'
```

```
print(mdf2)
```

```
# right=df2 기준으로 합침
```

```
mdf3 = pd.merge(left=df, right=df2, how='right', on='이름')
```

```
print(mdf3)
```

```
# df, df2의 모든 열 합침
```

```
mdf4 = pd.merge(left=df, right=df2, how='outer', on='이름')
```

```
print(mdf4)
```

# 2차원 데이터

```
data = {
```

```
    '이름':['주바다','공유진','송선유','양홍욱'
```

```
],
```

```
    '키':[180,182,188,179],
```

```
    '몸무게':[47,49,50,45]
```

```
}
```

```
data2={
```

```
    '이름':['주바다','공유진','송선유','윤상운'
```

```
],
```

```
    '전공':['컴퓨터공학','매카닉공학','독일
```

```
어학','컴퓨터공학'],
```

```
    '실력':['고급','중고급','특급','중고급']
```

Merge 기준이 되는 열로 합침

```
# inner - 동일한 열기준만 합침
```

```
# left - 왼쪽의 열 기준으로 합침
```

```
# right - 오른쪽 열 기준으로 합침
```

```
# outer - 모든 열 기준으로 합침
```

## 16 url txt파일 읽어오기

```
import pandas as pd

url =
'https://race.kra.co.kr/dbdata/fileDownload.do?fn=internet/seoul/horse/20220501sdb1.txt&me
et=1'

df = pd.read_csv(url,encoding='euc-kr')

print(df)
```

# url text파일 읽어오기  
url주소를 입력하면 됨.

# 저장된 파일 읽어오기  
df =  
pd.read\_csv('score.txt',encoding='utf-8')

# Quiz 1

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터  
1. 영화정보만 출력하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
Print(df)
```

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방
의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
print(df)

print(df['영화'])
```

```
## 개봉 연도에 년도를 추가
df['개봉 연도'] = df['개봉 연도'].astype(str) + "년도"
```

```
### 관객수 천단위 표시 해서 출력하시오.
df['관객 수'] = df['관객 수'].map('{:,d}'.format)
```

# 실습 과제

## 영화

# Quiz 2

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터  
1. 전체 데이터 중에서 '영화', '평점' 정보를 출력하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
Print(df)
```

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
data = {
    '영화': ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방
의 선물', '암살'],
    '개봉 연도': [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수': [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점': [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
print(df)

print(df[['영화', '평점']])
```

### 관객수 천단위 표시 해서 출력하시오.

df['관객 수'] = '\$' + df['관객 수'].map('{:,d}'.format)

### \$ 없애고, 천단위 표시 없애고, int타입으로 변경하시오.

df['관객 수'] = df['관객 수'].str[1:].str.replace(",", "").astype(int)

# 실습 과제

## 영화

[[ ]] 2개가 들어가야 함.

# Quiz 3

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

1. 2015년 이후에 개봉한 영화 데이터 중에서  
'영화', '개봉연도' 정보를 출력하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
Print(df)
```

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
print(df)
```

# 퀴즈3. 정답

```
print(df.loc[df['개봉 연도'] >= 2015, ['영화', '개봉 연도']])
```

실습  
과제

영화

# Quiz 4

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

1. 주어진 계산식을 참고하여  
'추천 점수' Column을 추가하시오.

[ 추천점수 계산 방법 ]

추천 점수 = (관객수 \* 평점) // 100

예) 첫 번째 영화인 '명량'의 경우,

추천 점수 = (관객수 1761 \* 평점 8.88) // 100 = 156

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
print(df)
```

# 퀴즈4. 정답

```
df['추천 점수'] = (df['관객 수'] * df['평점']) // 100
print(df)
```

# 실습 과제

## 영화

# Quiz 5

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

1. 전체 데이터를 '개봉 연도' 기준  
내림차순으로 출력하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
Print(df)
```

# Quiz 5

대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

1. 개봉연도 내림차순, 오름차순 으로 출력하시오.
2. 평점이 9점이상만 출력하시오.
3. 평점 컬럼 내용에 8.88점 글자를 넣어서 출력하시오. 함수적용  
(\*적용하지 않은 것 astype )
4. 평점중에 평점 평균이상인 row만 출력하시오.

# 15. 퀴즈

# 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터

```
import pandas as pd
```

```
data = {
```

```
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방  
의 선물', '암살'],
```

```
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
```

```
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
```

```
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

# 퀴즈5. 정답

```
print( df.sort_values('개봉 연도', ascending=False) )
```

# 실습 과제

## 영화

## 8 오라클 db 환경설정 - connection

데이터 분석 라이브러리 설치  
대체 : `python -m pip install oracledb`  
3.10 이전 버전 : `pip install cx_Oracle`

```
import oracledb as ora
```

```
#오라클 connection 연결
```

```
## conn = ora.connect('localhost:1521/x')
```

```
conn = connections()
```

```
print(conn.version)
```

```
# db연결부분을 함수로 정의
```

```
def connections():
```

```
    try:
```

```
        conn = oracledb.connect(user="ora_user", password="1111",  
                                dsn="localhost:1521/x")
```

```
# DB에 연결 (호스트이름 대신 IP주소 가능)
```

```
    except Exception as e:
```

```
        msg="예외발생"
```

```
        print(msg)
```

```
    return conn
```

## Views.py

# 오라클 연결

## import cx\_Oracle as ora

def ora\_list(request):

## 오라클 연결

conn = connections()

print(conn.version)

# cursor 할당

cursor = conn.cursor()

#SQL문 작성

sql\_insert = 'insert into item\_code values(:no,:name)'

# sql\_insert

# 입력값을 쿼리에 바인딩한 후 전송

cursor.execute(sql\_insert,no='0001',name='종목명')

#커서 닫고 commit 해준다.

cursor.close()

conn.commit()

# conn.close() # 리소스를 많이 잡혀 열어놓고 사용하기도 함.

return render(request, 'ora\_list.html')

# db연결부분 함수로 정의

def connections():

try:

conn= ora.connect('c##ora\_user1/1111@localhost:1521/xs')

cursor = conn.cursor()

except Exception as e:

msg="예외발생"

print(msg)

return conn

# 1개의 데이터 입력

# sql 구문 , board\_seq.nextval : 시퀀스 증가, sysdate : 날짜자동입력

sql\_insert = 'insert into board

values( board\_seq.nextval, :b\_id, :b\_title, :b\_content, sysdate )'

# 실행구문 - b\_no='1': 직접입력가능

cursor.execute( sql\_insert, b\_no=b\_no, b\_title=b\_title,

b\_content=b\_content)

print(qs.query) # sql쿼리문 확인

## 8 오라클 db연결 - execute 여러 row 저장, 저장타입

### Views.py

```
# 오라클 연결 여러개 파일
def ora_list(request):
    ## import cx_Oracle as ora
    ## 오라클 연결
    df =
    pd.read_excel('C:\WpyFolder\Wjs_work\score.xlsx', usecols='A:D', dtype={0:str, 1:str, 2:str, 3:str})
    df1 = df[['지원번호', '이름', '학교', '키']]
    rows = [ tuple(x) for x in df1.to_records(index=False) ]

    conn = connections()
    print(conn.version) # 18.4.0.0.0
    cursor = conn.cursor()
    print(cursor) # <cx_Oracle.Cursor on <cx_Oracle.Connection to
    ##ora_user1@localhost:1521/xes>

    # oracle db 에 저장
    cursor.execute('delete from score')
    score_query = 'insert into score values(:1,:2,:3,:4)'
    cursor.executemany(score_query, rows)
    conn.commit()

    cursor.execute('select count(*) from score')
    cnt = cursor.fetchone()
    print('insert 건수 : ', cnt)
    cursor.close()

    return render(request, 'notice_list.html')
```

# pandas 엑셀 데이터 불러오기

# usecols='A:D' # 엑셀컬럼 A,B,C,D를 가져옴.

# dtype = { 0:str, 1:str, 2:str, 3:str } # 0번째 컬럼 str타입 설정  
# 예) {0: float, 1:int}

# 1줄 for문은 리턴이 자동으로 LIST가 됨.

# rows는 tuple(x)형태의 LIST  
data=[  
 ('1번', '강나래', '구로고', '197'), ('2번', '강태원', '구로고', '184')  
 ...  
]

# 데이터 insert, select시 명령어  
cursor.execute(score\_insert, stu\_no='1번', stu\_name='강나래')  
cursor.executemany(score\_query, rows)

# 데이터 select시 명령어 -> execute후 fetch사용  
# cursor.execute(score\_select, stu\_no='1번')  
cursor.fetchall() : 모든 데이터를 가져올 때 사용  
cursor.fetchone() : 한번 호출에 1개 Row를 가져올 때 사용  
# 컬럼이 지원번호, 이름, 학교, 키 # no = row[0] # 지원번호 데이터 넘어옴  
# name = row[1] # 이름 넘어옴...  
# school = row[2]  
cursor.fetchmany(n) : n개 만큼의 데이터를 가져올 때 사용

## 8 오라클 db연결 - int일때 error

### Views.py

```
# 오라클 연결 여러개 파일
def ora_list(request):
    ## import cx_Oracle as ora
    ## 오라클 연결
    df =
    pd.read_excel('C:\WpyFolder\Wjs_work\score.xlsx', usecols='A:D', dtype={0: str, 1: str, 2: str, 3: str})
    df1 = df[['직원번호', '이름', '학교', '키']]
    rows = [ tuple(x) for x in df1.to_records(index=False) ]
    conn = connections()

    ## 이하동일
```

# pandas 엑셀 데이터 불러오기

# usecols='A:D' # 엑셀컬럼 A,B,C,D를 가져옴.

# dtype = { 0:str, 1:str, 2:str, 3:str } # 0번째 컬럼 str타입 설정  
# 예) {0: float, 1:int}

# 데이터를 tuple()형태의 list타입으로 데이터 저장.

```
# rows = [ tuple(x) for x in df.to_records(index=False) ]
rows=[]
for x in df.to_records(index=False):
    row=[x[0],x[1],x[2],int(x[3])]
    rows.append(row)
```

파이썬 pandas 를 이용하여 str 변수는 문제가 없으나  
int인 경우는 변환 에러 발생

(해결) int타입으로 한번 더 변경해 줌.

## 8 오라클 db연결 - select 데이터 가져오기

```
import cx_Oracle
```

```
# rowfactory 를 통해 데이터 타입 변경
```

```
def makeDictFactory(cursor):  
    columnNames = [d[0] for d in cursor.description]
```

```
    def createRow(*args):  
        return dict(zip(columnNames, args))
```

```
    return createRow
```

```
# 오라클 db연결
```

```
conn = connections()  
cursor = conn.cursor()  
cursor.execute("select * from test_tbl where rownum < 2")
```

```
cursor.rowfactory = makeDictFactory(cursor)
```

```
rows = cursor.fetchall()
```

```
for row in rows :  
    print(row)
```

```
conn.close()
```

# 다행히 cxOracle은 cursor의 rowfactory 라는 메서드를  
오버라이딩하여 리턴받는 데이터의 형태를 바꿀 수 있음.

```
{'name': 'kim', 'age': 27, 'address': '서울시 광진구'}
```

```
name = row['name'] #kim  
age = row['age'] #27  
address = row['address'] #서울시 광진구
```

# cxOracle 라이브러리는 SELECT 해오는 쿼리 결과 : tuple형태 list타입

```
('kim', 27, '서울시 광진구')
```

```
name = row[0] #kim  
age = row[1] #27  
address = row[2] #서울시 광진구
```

# 데이터를 확인

```
name = row['name'] #kim  
age = row['age'] #27  
address = row['address'] #서울시 광진구
```

## 8 오라클 db연결 - DataFrame 으로 받기

```
import cx_Oracle
import pandas as pd
from stboard.models import board
```

```
# db연결함수로 정의
def connections():
    try:
        conn= ora.connect('system/1234@localhost:1521/xe')
        # cursor = conn.cursor()
    except Exception as e:
        msg="예외발생"
        print(msg)
    return conn
```

```
# pd. Read_sql_query 구문 사용
def blist(request):
    query ='select * from board'
    df = pd.read_sql_query(query,connections())
    print(df)
```

```
# pandas 라이브러리 read_sql_query사용
# sql에서 바로 DataFrame만듬.
```

THANK YOU